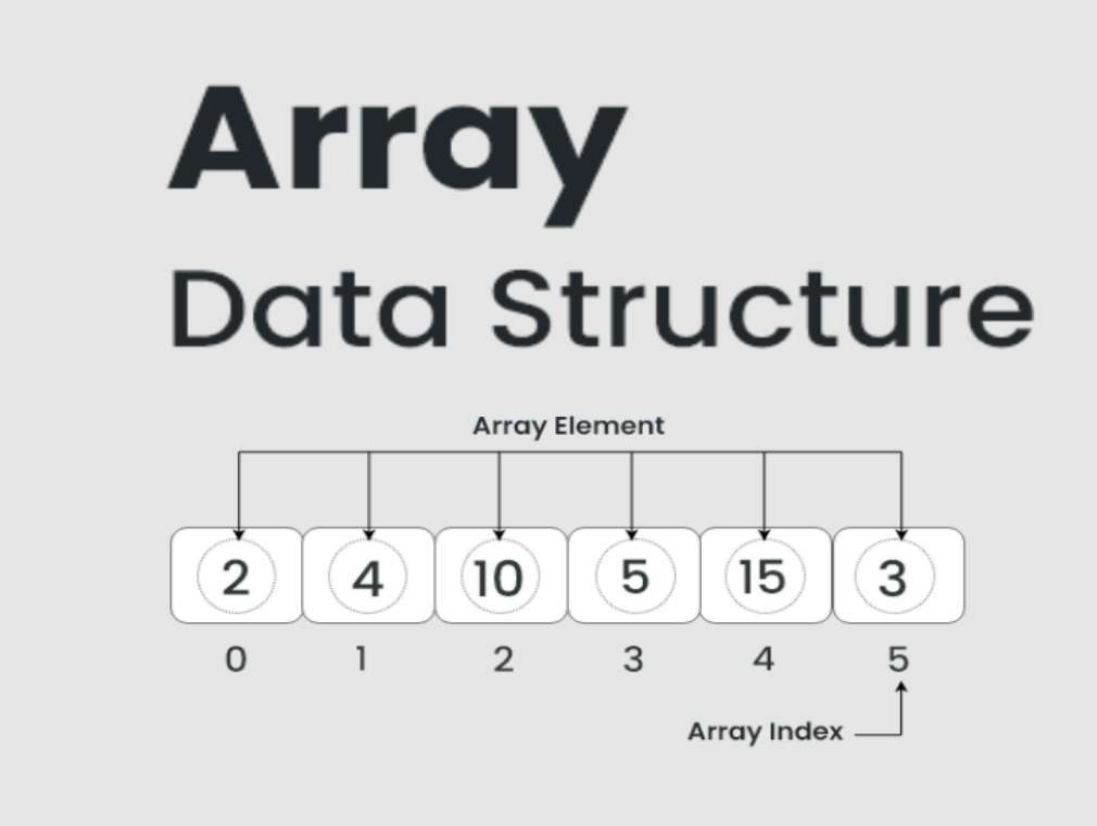
# Fundamental Data Structure




**Arrays** are a fundamental data structure used to store multiple values of the **same type** in a **single variable.** They have a fixed size and allow indexed access to elements**.**

**Example:-**

```
int [ ] numbers = {10, 20, 30, 40, 50};
String[ ] fruits = {"Apple", "Banana", "Orange"};
Int[ ] numbers = new int[5];
```

In Java, Arrays can store **different types of data**, but the **type must be consistent within the array.**

```
int [ ] numbers = {1, 2, 3, 4, 5};
Object[ ] data = new Object[5];
data[0] = "Hello"; → String
data[1] = 100;  →Integer
data[2] = 45.6; → Double
```

```
data[3] = new Person("John", 28);
data[4] = true; → Boolean
```

## Limitations of arrays in Java

### 1. Fixed Size

- Once an array is created, its size **cannot be changed.**
- You must know the size in advance.
- **Problem:** If you need more space, you must create a new array and copy elements.

### 2. Homogeneous Data

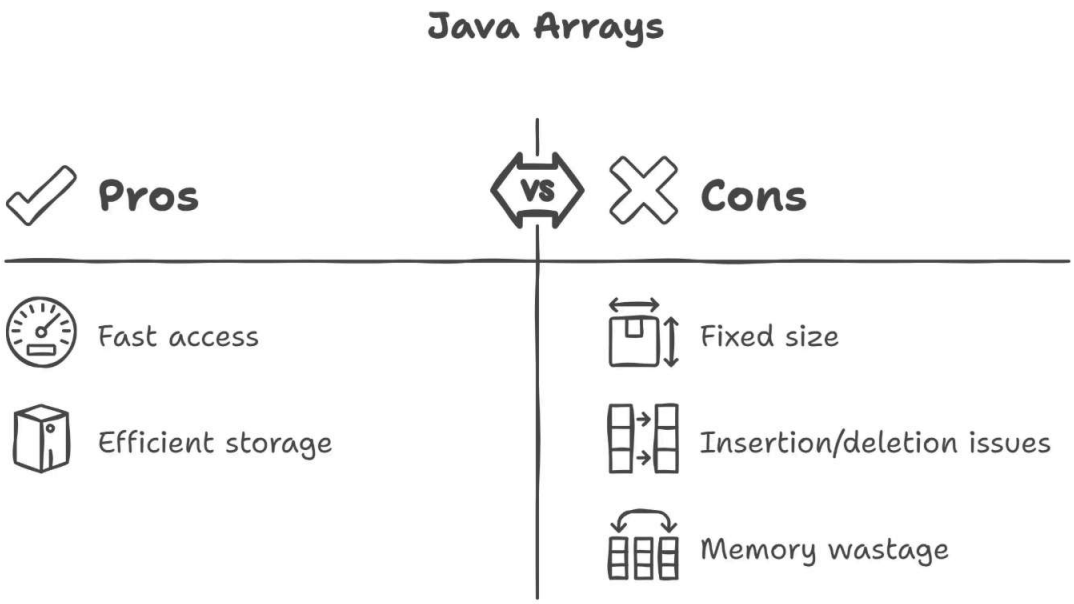
- Arrays can only store elements of the **same data type** (except Object[] arrays).
- **Problem:** Cannot mix primitive types easily (without Object array, which requires casting).

### 3. No Built-in Methods for Manipulation

- Arrays lack **rich utility methods** like adding, removing, or searching elements.

### 4. Memory Waste (for Large Arrays)

- If you declare a large array but use only a few elements, **memory is wasted.**
- Fixed size can lead to **under-utilization.**




# Introduction to Collection

## Collection in Java:

A collection is a group of individual data that are grouped to form a single unit. Collections are containers that hold multiple data items as a single unit. For example, if we store the names of all the employees in a single list and name it Employee, it will form a collection.
The two root interfaces of java collection class are:

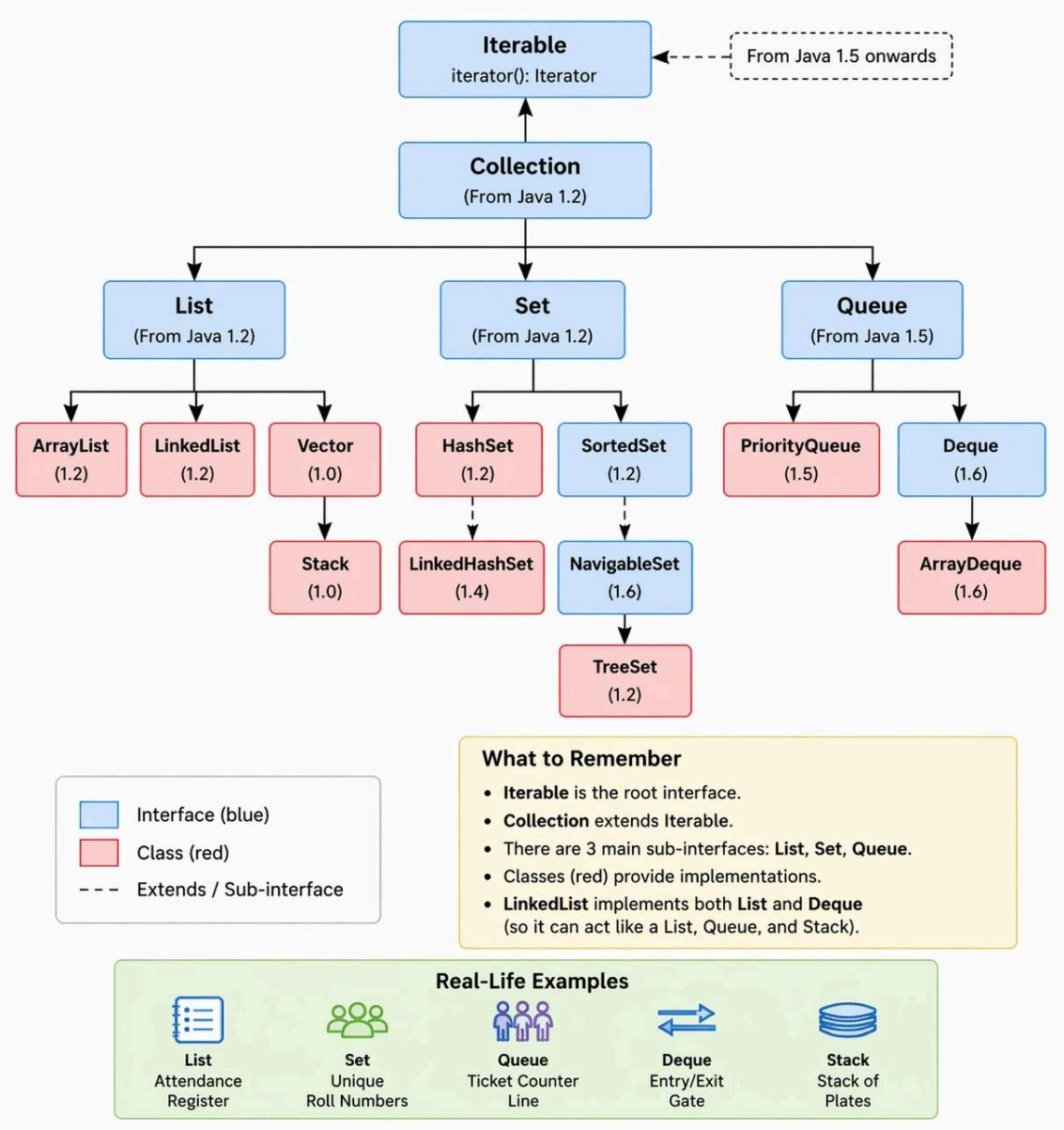
- Collection interface(java.util.collection)
- Map interface(java.util.Map)

## Collection Framework in Java:

The **Java Collection Framework (JCF)** is a **set of classes and interfaces** that provide **predefined data structures** to store, retrieve, and manipulate groups of objects efficiently. It is part of the **java.util package**.

## Hierarchy of Collection Framework in Java:

As we know, the collection framework is an architecture of classes and interfaces. Let us now see their hierarchy.




## Why Use Collection Framework?

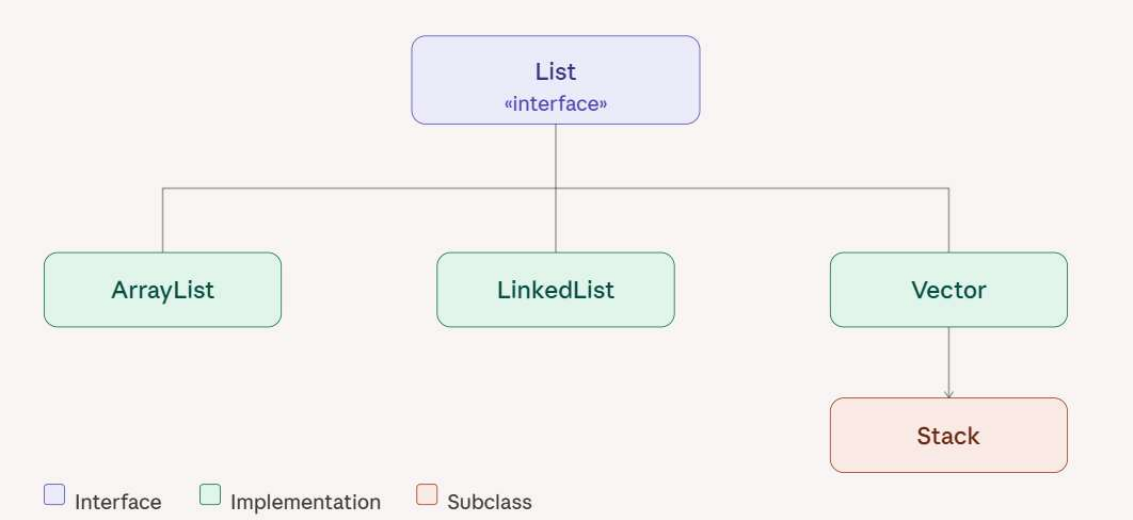
- Arrays have a **fixed size** and limited functionality.
- Collection Framework provides **dynamic, flexible, and powerful ways** to store and manipulate objects.
- Reduces **programming effort** by providing ready-to-use data structures.

The **Collection interface** is the root interface in the **Java Collection Framework**, from which most other collection interfaces (List, Set, Queue) inherit. It provides **general methods to work with groups of objects**.

## 1. Basic Methods of Collection interface

| Method | Description |
| --- | --- |
| boolean add(E e) | Adds the specified element to the collection. |
| boolean addAll(Collection<? extends E> c) | Adds all elements from the specified collection. |
| void clear() | Removes all elements from the collection. |
| boolean contains(Object o) | Returns true if the collection contains the specified element. |
| boolean containsAll(Collection<?> c) | Returns true if the collection contains all elements of the specified collection. |
| boolean isEmpty() | Returns true if the collection has no elements. |
| boolean remove(Object o) | Removes a single instance of the specified element, if present. |
| boolean removeAll(Collection<?> c) | Removes all elements that are present in the specified collection. |
| boolean retainAll(Collection<?> c) | Retains only elements that are present in the specified collection. |
| int size() | Returns the number of elements in the collection. |

# List Interface




The **List interface** is a **subinterface of Collection** in Java. It represents an **ordered collection** of elements where:

- **Duplicates are allowed –** you can store the same element multiple times.
- **Insertion order is preserved –** elements are stored and retrieved in the order they were added.
- **Indexed access –** elements can be accessed, inserted, or removed using their **position (index).**

The List interface is part of the **java.util package.**

## Common List Methods

Key methods for manipulating and inspecting lists include:

- **Modification**: add(), set(), remove(), and clear() manage elements.
- **Access/Information:** get(), size(), isEmpty(), and contains() query the list.
- **Search/Order:** indexOf(), lastIndexOf(), and sort() allow searching and ordering elements.
- **Bulk Operations:** addAll(), removeAll(), and retainAll() allow operations on collections.
- **View/Conversion:** subList(), toArray(), iterator(), and listIterator() provide ways to view or traverse the data.

# Example with Storing with heterogeneous collection:-

## Example 1: Using ArrayList<Object>

```java
import java.util.*;
public class HeterogeneousListExample {
  public static void main(String[] args) {
    List<Object> list = new ArrayList<>();

    list.add("Hello");      // String
    list.add(100);          // Integer
    list.add(45.67);        // Double
    list.add(true);         // Boolean
    list.add('A');          // Character

    for (Object obj : list) {
      System.out.println(obj);
    }
  }
}
```

## Example 2: With Custom Objects + Primitive Wrappers

```java
import java.util.*;

class Student {
  String name;
  Student(String name) {
    this.name = name;
  }
  public String toString() {
    return name;
  }
}

public class MixedDataExample {
  public static void main(String[] args) {
    List<Object> list = new ArrayList<>();
```

```
        list.add(new Student("Rahul")); // Custom object
        list.add(500);                  // Integer
        list.add("Java");               // String

        for (Object obj : list) {
            System.out.println(obj);
        }
    }
}
```

## Important Notes

- Java collections are **generic**, so using List<Object> allows mixed types.
- Type safety is **lost**, so you may need casting when retrieving elements:

```
Object obj = list.get(0);
if (obj instanceof String) {
    String str = (String) obj;
}
Alternative: Using Raw Types (Not Recommended)
List list = new ArrayList(); // raw type

list.add("Hello");
list.add(10);
list.add(3.14);
```

This works but **avoids generics**, so it's not type-safe and should be avoided in modern Java.

## Example with Storing with homogeneous collection using Generics:-

```
public class ListExample {
public static void main(String[] args) {
        List<String> fruits = new ArrayList<>();
```

```
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Orange");
        fruits.add("Apple"); // duplicates allowed
        System.out.println("Fruits: " + fruits);
         // Access by index
        System.out.println("First fruit: " + fruits.get(0));
        // Insert at index
        fruits.add(1, "Mango");
        System.out.println("After insertion: " + fruits);
        // Remove by index
        fruits.remove(2);
        System.out.println("After removal: " + fruits);
    }
}
```

## 1. ArrayList

ArrayList is a **resizable array** implementation of the List interface.

- **Duplicates allowed**
- **Maintains insertion order**
- **Dynamic size –** grows automatically when needed
- **Fast random access** using an index

### Real-life example

Think of a **playlist of songs** in your music app. You can add, remove, or access songs by their position easily.

Code Example

```
public class ArrayListExample {
    public static void main(String[] args) {
        List<String> playlist = new ArrayList<>();
        // Adding songs
        playlist.add("Shape of You");
        playlist.add("Blinding Lights");
        playlist.add("Believer");
        playlist.add("Shape of You"); // duplicates allowed
        System.out.println("Playlist: " + playlist);
        // Access by index
        System.out.println("First song: " + playlist.get(0));
```

```
        // Insert at index
        playlist.add(1, "Levitating");
        System.out.println("After insertion: " + playlist);
    }
}
```

## 2. LinkedList

LinkedList is a **doubly linked list** implementation of the List and Deque interfaces.

- Maintains **insertion order**
- Allows **duplicates**
- **Efficient insertion/deletion** in the middle

### Real-life example

Think of a **train**. Each compartment is linked to the next and previous one. Adding or removing a compartment is easy without shifting the others.

```
public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> train = new LinkedList<>();
// Adding compartments
        train.add("Engine");
        train.add("Coach1");
        train.add("Coach2");
        train.add("Coach3");
        System.out.println("Train: " + train);
// Adding at first and last
        train.addFirst("Engine-1");
        train.addLast("Coach4");
        System.out.println("Updated Train: " + train);
        // Removing a compartment
        train.remove("Coach2");
        System.out.println("After removal: " + train);
    }
}
```

## 3. Vector

Vector is similar to ArrayList but **synchronized** (thread-safe).

- Dynamic array
- Maintains insertion order
- Allows duplicates

### Real-life example

Think of a **bank account transaction log** where multiple threads might add transactions at the same time. Synchronization ensures **thread safety.**

```
public class VectorExample {
  public static void main(String[] args) {
     Vector<String> transactions = new Vector<>();
     transactions.add("Deposit $100");
     transactions.add("Withdraw $50");
     transactions.add("Deposit $200");
     System.out.println("Transactions: " + transactions);
     // Access by index
     System.out.println("First transaction: " + transactions.get(0));
   }
}
```

## 4. Stack

A stack is a **last-in-first-out (LIFO)** data structure.

- Extends Vector
- Operations: push(), pop(), peek()

### Real-life example

Think of a **stack of plates in a cafeteria.** You put a new plate on top (push) and remove the top plate first (pop).

```
public class StackExample {
  public static void main(String[] args) {
     Stack<String> plates = new Stack<>();
// Adding plates
     plates.push("Plate1");
     plates.push("Plate2");
```

```
        plates.push("Plate3");
        System.out.println("Plates stack: " + plates);
    // Remove top plate
        String removed = plates.pop();
        System.out.println("Removed: " + removed);
        System.out.println("After pop: " + plates);
        // View top plate without removing
        System.out.println("Top plate: " + plates.peek());
    }
}
```

# Iterator and ListIterator in Java

## Iterator

An **Iterator** is an interface in the **Java Collection Framework** used to **traverse (iterate) elements** of a collection **one by one**.

**It belongs to:**

java.util package

**Iterator allows:**

✓ Forward traversal
✓ Safe element removal
✓ Generic collection traversal

**Why is an Iterator Needed?**

**Before Iterator:**

Traversal depended on the collection type.
Different collections needed different logic.

**Iterator provides:**

Unified way to traverse collections.

**Works with:**

- ArrayList
- LinkedList

- HashSet
- TreeSet
- HashMap (via entrySet)

**Iterator Interface Methods**

```
public interface Iterator<E> {

    boolean hasNext();
    E next();
    void remove();
}
```

**Method 1 — hasNext()**

Checks whether the next element exists or not.

**Returns:**

true → if element exists
false → if no element

**Method 2 — next()**

Returns the next element, moves the iterator pointer forward.

**Method 3 — remove()**

Removes the last returned element. Used for **safe removal during iteration**.

## Iterator Working Logic

Collection → Iterator → Elements

```
[10, 20, 30]
```

Pointer starts before the first element.

**Steps:**

- hasNext() → true
- next() → returns element
- Repeat

**Example 1 — Iterator with ArrayList**

```
import java.util.ArrayList;
import java.util.Iterator;
```

```
public class IteratorExample1 {
   public static void main(String[] args) {

      ArrayList<String> list = new ArrayList<>();
      list.add("Java");
      list.add("Python");
      list.add("C++");

      Iterator<String> itr = list.iterator();

      while (itr.hasNext()) {
         String language = itr.next();
         System.out.println(language);
      }
   }
}
```

Output:-

```
Java
Python
C++
```

**Example 2— Removing Elements Using Iterator**

```
import java.util.ArrayList;
import java.util.Iterator;
public class IteratorRemoveExample {
   public static void main(String[] args) {

      ArrayList<Integer> list = new ArrayList<>();

      list.add(10);
      list.add(20);
      list.add(30);
      list.add(40);

      Iterator<Integer> itr = list.iterator();
      while (itr.hasNext()) {
         Integer num = itr.next();
         if (num == 20) {
```

```
                itr.remove(); // Safe removal
            }
        }
        System.out.println(list);
    }
}
Output:-
[10, 30, 40]
```

## Pitfall — Removing Without Iterator

**Wrong:**

```
for(Integer num : list) {
    if(num == 20) {
        list.remove(num); // Exception
    }
}
```

**Runtime Error:**

ConcurrentModificationException

**Correct:**

```
itr.remove();
```

## Limitations of Iterator:-

**Iterator supports:**

- Forward traversal

**Iterator does NOT support:**

- Backward traversal
- Index access
- Element replacement

## ListIterator in Java

**ListIterator** is a specialized iterator used only with **List collections.**

**Works with:**

- ArrayList
- LinkedList
- Vector

Does NOT work with:

- Set
- Map

### ListIterator Interface Methods

```
public interface ListIterator<E>
      extends Iterator<E> {
   boolean hasPrevious();
   E previous();
   int nextIndex();
   int previousIndex();
   void set(E e);
   void add(E e);
}
```

**Example 1— Forward Traversal**

```
import java.util.ArrayList;
import java.util.ListIterator;
public class ListIteratorForward {
   public static void main(String[] args) {

      ArrayList<String> list = new ArrayList<>();
      list.add("A");
      list.add("B");
      list.add("C");

      ListIterator<String> litr = list.listIterator();

      while (litr.hasNext()) {
         System.out.println(litr.next());
      }
   }
}
```

**Example 2 — Backward Traversal**

```
import java.util.ArrayList;
import java.util.ListIterator;
public class ListIteratorBackward {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("Java");
      list.add("Python");
      list.add("C++");

      ListIterator<String> litr =  list.listIterator(list.size());

      while (litr.hasPrevious()) {
         System.out.println(
               litr.previous());
      }
   }
}
```

Output:-

```
C++
Python
Java
```

**Example 3— Modifying Elements**

```
import java.util.ArrayList;
import java.util.ListIterator;
public class ListIteratorModify {
   public static void main(String[] args) {

      ArrayList<String> list = new ArrayList<>();
      list.add("java");
      list.add("python");

      ListIterator<String> litr =  list.listIterator();

      while (litr.hasNext()) {
         String lang = litr.next();
```

```
            litr.set(lang.toUpperCase());
        }
        System.out.println(list);
    }
}
```

Output:-

[JAVA, PYTHON]

**Example 4— Adding Elements**

```
import java.util.ArrayList;
import java.util.ListIterator;
public class ListIteratorAdd {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(30);
        ListIterator<Integer> litr = list.listIterator();
        while (litr.hasNext()) {
            int num = litr.next();
            if (num == 10) {
                litr.add(20);
            }
        }
        System.out.println(list);
    }
}
```

Output:-

[10, 20, 30]

## Iterator vs ListIterator

| Feature | Iterator | ListIterator |
|---|---|---|
| Package | java.util | java.util |
| Forward traversal | ✓ | ✓ |

| | | |
|---|---|---|
| Backward traversal | ❌ | ✓ |
| Add element | ❌ | ✓ |
| Remove element | ✓ | ✓ |
| Replace element | ❌ | ✓ |
| Works on List only | ❌ | ✓ |

### When to Use What?

**Use Iterator when:**

- Traversing Set
- Simple forward traversal

**Use ListIterator when:**

- Need backward traversal
- Need modification
- Working with List

## Adding primitives to a Collection:

**Note:** All the collection classes do not allow the primitive types; they only accept objects.

So, if we want to store any primitive values, we need to store them in the form of corresponding wrapper classes. Internally, collection classes use auto-boxing and auto-unboxing features to store the primitive in the form of corresponding wrapper class objects.

Example:

```
ArrayList<Integer> al = new ArrayList<>();
al.add(10);
al.add(12);
```

## Auto boxing and Auto unboxing:

The automatic conversion of primitive data types into their equivalent Wrapper type is known as boxing, and the opposite operation is known as unboxing. This is the new feature of Java 5. So a Java programmer doesn't need to write the conversion code.

The main advantage of Autoboxing and unboxing is, no need of conversion between primitives and Wrappers manually, so less coding is required.

### Example

```
int x = 10;
//let's convert this primitive to the corresponding wrapper object
Integer i1 = Integer.valueOf(x); // boxing
Integer i2 = x; //autoboxing
or
Integer i3 = 10; //autoboxing

Unboxing:

int x = i3.intValue(); //unboxing
int x = i3; // auto-unboxing
```

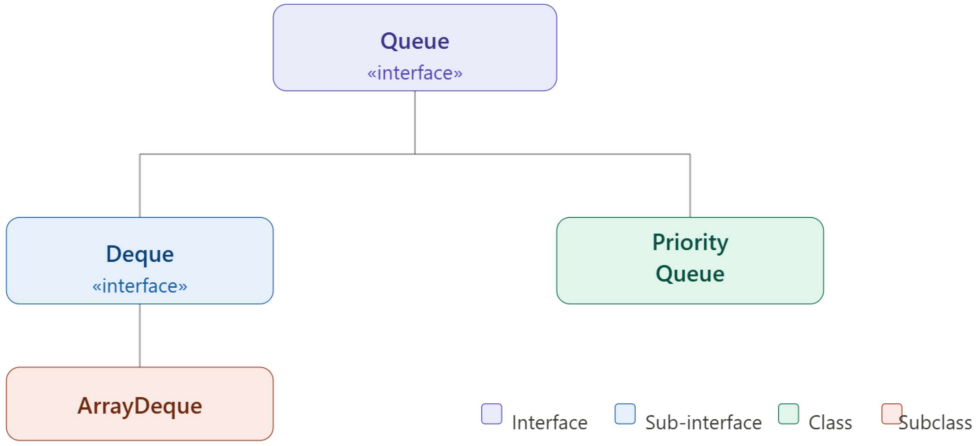
### Example:

```
import java.util.ArrayList;

class Main {
   public static void main(String args[]) {
      ArrayList<Integer> al = new ArrayList<>();
      al.add(10);
      al.add(20);
      al.add(30);
      al.add(40);
      al.add(50);
      for(int x: al){
         System.out.println(x);
      }
   }
}
```

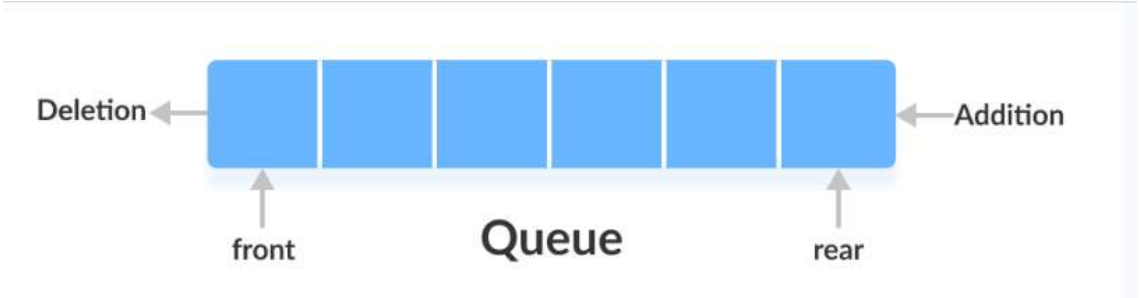
Note: We can call all the methods defined inside the Collection interface on the ArrayList object.

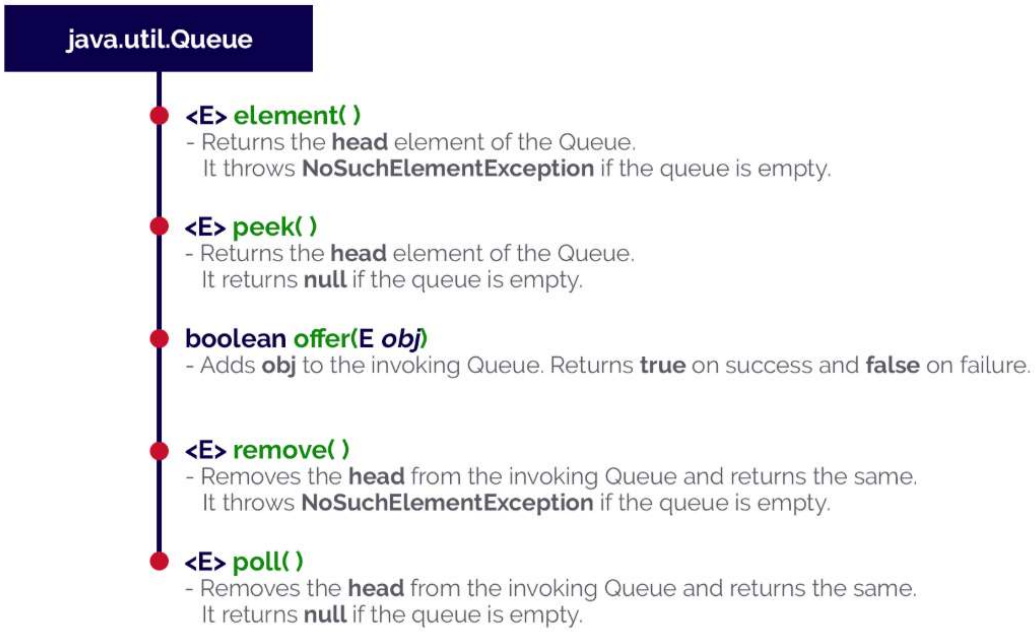
# Queue Interface




A Queue is a data structure that follows FIFO (First In, First Out), i.e. The element added first is removed first.

**Example:** Think of it like:
A line at a ticket counter
First person in line = first served.






## 1. Priority Queue :

A PriorityQueue in Java is a queue where elements are ordered based on their priority, rather than the order of insertion. By default, it uses natural ordering (min-heap), but a custom comparator can be used to define different priorities.

```
Queue<Integer> pq = new PriorityQueue<>();
pq.add(30);
pq.add(10);
pq.add(20);
System.out.println(pq); // Not sorted output
while(!pq.isEmpty()) {
  System.out.println(pq.poll());
}
```

O/P→10,20,30 Automatically sorted (min-heap by default)

## 2. ArrayDeque :

An *ArrayDeque* (also known as an “Array Double Ended Queue”, pronounced as “ArrayDeck”) is a special kind of growable array that allows us to add or remove an element from both sides.

An *ArrayDeque* implementation can be used as a *Stack* (Last-In-First-Out) or a *Queue*(First-In-First-Out).

```
Queue<Integer> dq = new ArrayDeque<>();
dq.offer(1);
dq.offer(2);
dq.offer(3);
```

```
System.out.println(dq.poll()); // 1
```

## Coding Tasks :

**1. Basic List Operations  Write a Java program to:**

- Add 5 integers to a list
- Print the list
- Remove the 3rd element
- Update the 2nd element
- Check if a number exists in the list

**2. Student Management System**

Create a class Student with: id, name, marks

Create a class StudentManager that:

- Stores students using a List
- Adds a student
- Removes a student by ID
- Displays all students
- Finds topper student

Use ArrayList internally

**3. Movie Booking System (Queue)**

Create a class User: name, userId

Create a class BookingQueue:

- Add user to the queue (book ticket request)
- Process booking (remove from queue)
- Show next user

Follow FIFO using a queue

## Difference Between ArrayList and LinkedList

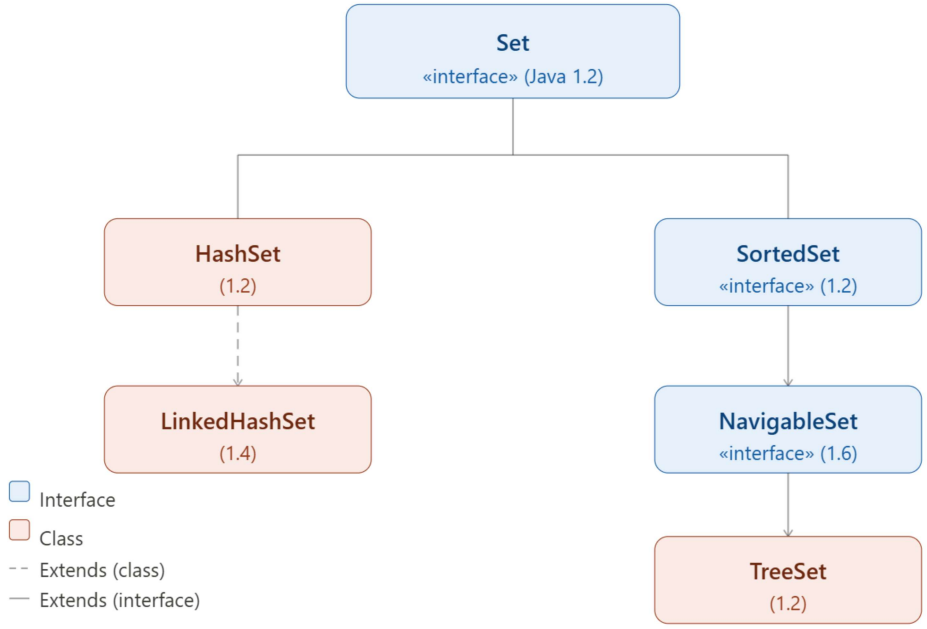
| ArrayList | LinkedList |
|---|---|
| Uses a dynamic array to store its elements | Uses a doubly-linked list, where each element is a node |
| Provides fast random access (O(1)) to elements using an index | Requires sequential traversal to find an element, making access slow (O(n)) |
| Insertion/deletion in the middle of the list is slow (O(n)) | Insertion and deletion at any position are fast (O(1)) |
| Lower memory overhead per element, as it only stores the data | Higher memory as each node stores not only the data but also two pointers |

| STACK | QUEUE |
|---|---|
| It represents the collection of elements in Last in first out(FIFO) | It represents the collection of elements in first in first out(FIFO) |
| Objects are inserted and removed at the same end called Top of the stacks. | Objects are inserted and removed from different ends called front and rear ends. |
| Insert operation is called Push Operation. | Inset operation is called enqueue operation. |
| Delete operation is called POP operation. | Delete operation is called Dequeue. |
| In stack there is no wastage of memory scope. | In queue there is a wastage of memory space. |
| Plate counter at marriage reception is an example of stack. | Students standing in a line at fees counter is an example of queue. |

# Array Vs Arraylist in Java ?

| Array | ArrayList |
| --- | --- |
| Here we can't revise the length of the array once constructed. | In ArrayList, we can revise the length of the array. |
| An array can hold primitives and objects both in Java. | ArrayList can only hold objects, not primitives. |
| Array is static. | ArrayList is dynamic. |
| It can either be single-dimensional or multidimensional. | It can be only single-dimensional |
| Through the length keyword, we can determine the total size of an array. | Through the size() method, we can determine the size of an ArrayList. |
| It is faster than ArrayList due to its static behaviour. | It is slower as compared to the Array due to its dynamic behaviour. |

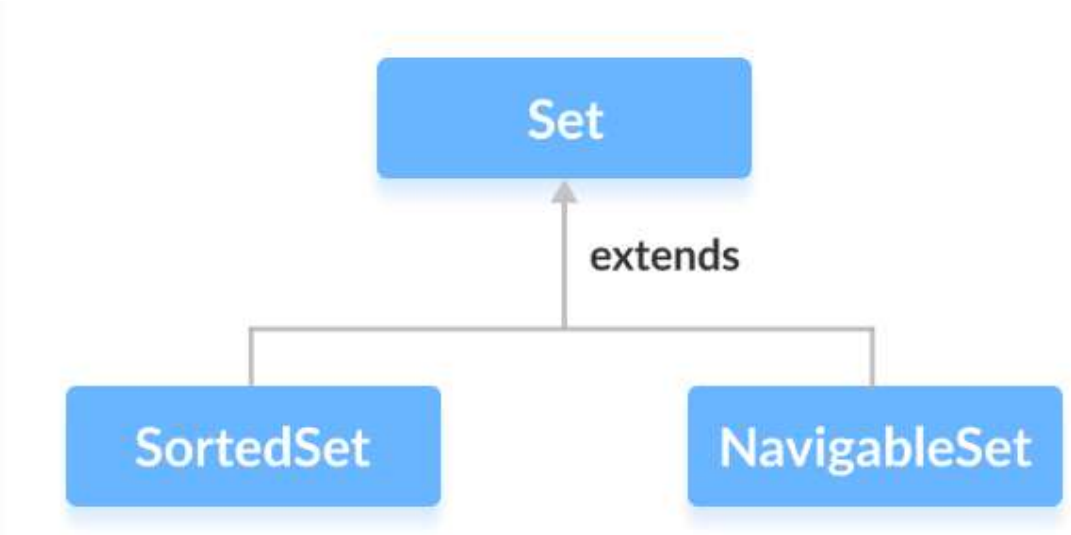
# SET




These classes are defined in the Collections framework and implement the Set interface.

The Set interface of the Java Collections framework provides the features of a mathematical set in Java. It extends the Collection interface. Unlike the List interface, sets cannot contain duplicate elements.

## The Set interface is also extended by these subinterfaces:




## Methods of Set

The Set interface includes all the methods of the Collection interface. It's because Collection is a super interface of Set.

Some of the commonly used methods of the Collection interface that are also available in the Set interface are:

- **add() -** adds the specified element to the set
- **addAll() -** adds all the elements of the specified collection to the set
- **iterator() -** returns an iterator that can be used to access elements of the set sequentially
- **remove() -** removes the specified element from the set
- **removeAll() -** removes all the elements from the set that is present in another specified set
- **retainAll() -** retains all the elements in the set that are also present in another specified set
- **clear() -** removes all the elements from the set
- **size() -** returns the length (number of elements) of the set
- **toArray() -** returns an array containing all the elements of the set
- **contains() -** returns true if the set contains the specified element
- **containsAll() -** returns true if the set contains all the elements of the specified collection
- **hashCode() -** returns a hash code value (address of the element in the set)

## Example -

```
import java.util.*;
public class Demo extends Practice {
```

```
    public static void main(String[] args) {
        // Creating an object of Set and declaring a String type
        Set<String> h = new HashSet<String>();
        // Adding elements to Set using add() method, Custom input elements
        h.add("A");
        h.add("B");
        h.add("C");
        // Removing element using remove() method
        h.remove("B");
        // Checking if an element exists using the contains() method
        h.contains("B");
        // Iterating through the Set via for-each loop
        for (String value : h) {
            System.out.println(value);
        }
    }
}
```

Output -

```
false
A
C
```

## HashSet Class

HashSet in Java implements the Set interface of the Collections Framework. It is used to store the unique elements, and it doesn't maintain any specific order of elements.

- HashSet does not allow duplicate elements.
- Uses HashMap internally, which is an implementation of the hash table data structure.
- Does not support primitive types directly; requires wrapper classes (Integer, Character, etc.).

**Example -**

```
import java.util.*;

public class demo {
    public static void main(String[] args) {
        // 1. Creating a HashSet
        HashSet<String> set = new HashSet<>();
```

```
// 2. add() - Adding elements
set.add("Apple");
set.add("Banana");
set.add("Mango");
set.add("Orange");
set.add("Apple"); // Duplicate (will not be added)
System.out.println("After adding elements: " + set);
// 3. size() - Number of elements
System.out.println("Size of set: " + set.size());
// 4. contains() - Check if element exists
System.out.println("Contains Mango? " + set.contains("Mango"));
// 5. remove() - Remove element
set.remove("Banana");
System.out.println("After removing Banana: " + set);
// 6. isEmpty() - Check if empty
System.out.println("Is set empty? " + set.isEmpty());
// 7. Iteration using a for-each loop
System.out.println("Using for-each loop:");
for (String fruit : set) {
    System.out.println(fruit);
}
// 8. Iteration using Iterator
System.out.println("Using Iterator:");
Iterator<String> it = set.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
// 9. clone() - Copy the set
HashSet<String> clonedSet = (HashSet<String>) set.clone();
System.out.println("Cloned set: " + clonedSet);
// 10. addAll() - Add another collection
HashSet<String> newSet = new HashSet<>();
newSet.add("Grapes");
newSet.add("Pineapple");
set.addAll(newSet);
System.out.println("After addAll(): " + set);
// 11. containsAll() - Check if all elements exist
```

```
        System.out.println("Contains all from newSet? " +
set.containsAll(newSet));
        // 12. removeAll() - Remove all elements from another set
        set.removeAll(newSet);
        System.out.println("After removeAll(): " + set);
        // 13. retainAll() - Keep only common elements
        HashSet<String> anotherSet = new HashSet<>();
        anotherSet.add("Apple");
        anotherSet.add("Kiwi");
        set.retainAll(anotherSet);
        System.out.println("After retainAll(): " + set);
        // 14. clear() - Remove all elements
        set.clear();
        System.out.println("After clear(): " + set);
        // 15. hashCode() - Returns the hash code of the set
        System.out.println("Hash code of set: " + set.hashCode());
    }
}
```

Output-

```
After adding elements: [Apple, Mango, Orange, Banana]
Size of set: 4
Contains Mango? true
After removing Banana: [Apple, Mango, Orange]
Is set empty? false
Using for-each loop:
Apple
Mango
Orange
Using Iterator:
Apple
Mango
Orange
Cloned set: [Mango, Orange, Apple]
After addAll(): [Apple, Grapes, Mango, Pineapple, Orange]
Contains all from newSet? true
```

```
After removeAll(): [Apple, Mango, Orange]
After retainAll(): [Apple]
After clear(): []
Hash code of set: 0
```

## LinkedHashSet -

LinkedHashSet in Java implements the Set interface of the Collections Framework.

- It combines the functionalities of a HashSet with a doubly-linked list to maintain the insertion order of elements.
- LinkedHashSet stores unique elements only and allows a single null.

### LinkedHashSet is a class in Java that:

- Implements the **Set interface**
- Stores **unique elements (no duplicates)**
- Maintains **insertion order**

### It is a combination of:

- Hash table (for fast operations)
- Linked list (to maintain order)

### Package & Syntax

```
import java.util.LinkedHashSet;

LinkedHashSet<Type> set = new LinkedHashSet<>();
```

### Features

- No duplicate elements allowed
- Maintains **insertion order**
- Allows **one null value**
- Not synchronized (not thread-safe)
- Faster than TreeSet, slightly slower than HashSet

### Internal Working

- Uses **HashMap internally**
- Each element is stored as a key
- A **doubly linked list** connects elements
- This linked list helps maintain order

**So:**

- Hashing → fast operations
- Linked list → order maintained

**Example -**

```
import java.util.*;

public class linkedhashset1 {
  public static void main(String[] args) {
    // 1. Creating LinkedHashSet
    LinkedHashSet<String> set = new LinkedHashSet<>();
    // 2. add() - Adding elements (insertion order maintained)
    set.add("Apple");
    set.add("Banana");
    set.add("Mango");
    set.add("Orange");
    set.add("Apple"); // Duplicate (ignored)
    System.out.println("After adding elements: " + set);
    // 3. size() - Number of elements
    System.out.println("Size: " + set.size());
    // 4. contains() - Check element
    System.out.println("Contains Mango? " + set.contains("Mango"));
    // 5. remove() - Remove element
    set.remove("Banana");
    System.out.println("After removing Banana: " + set);
    // 6. isEmpty() - Check if empty
    System.out.println("Is empty? " + set.isEmpty());
    // 7. Iteration using for-each (order preserved)
    System.out.println("Using for-each:");
    for (String fruit : set) {
      System.out.println(fruit);
    }
    // 8. Iterator - Another way to traverse
    System.out.println("Using Iterator:");
    Iterator<String> it = set.iterator();
    while (it.hasNext()) {
      System.out.println(it.next());
    }
```

```
        // 9. clone() - Copy set
        LinkedHashSet<String> clonedSet = (LinkedHashSet<String>)
set.clone();
        System.out.println("Cloned set: " + clonedSet);
        // 10. addAll() - Add another collection
        LinkedHashSet<String> newSet = new LinkedHashSet<>();
        newSet.add("Grapes");
        newSet.add("Pineapple");
        set.addAll(newSet);
        System.out.println("After addAll(): " + set);
        // 11. containsAll() - Check all elements
        System.out.println("Contains all newSet? " + set.containsAll(newSet));
        // 12. removeAll() - Remove all elements from newSet
        set.removeAll(newSet);
        System.out.println("After removeAll(): " + set);
        // 13. retainAll() - Keep only common elements
        LinkedHashSet<String> anotherSet = new LinkedHashSet<>();
        anotherSet.add("Apple");
        anotherSet.add("Kiwi");
        set.retainAll(anotherSet);
        System.out.println("After retainAll(): " + set);
        // 14. clear() - Remove everything
        set.clear();
        System.out.println("After clear(): " + set);
        // 15. hashCode() - Hash value of set
        System.out.println("HashCode: " + set.hashCode());
    }
}
```

Output -

```
After adding elements: [Apple, Banana, Mango, Orange]
Size: 4
Contains Mango? true
After removing Banana: [Apple, Mango, Orange]
Is empty? false
Using for-each:
Apple
Mango
```

```
Orange
Using Iterator:
Apple
Mango
Orange
Cloned set: [Apple, Mango, Orange]
After addAll(): [Apple, Mango, Orange, Grapes, Pineapple]
Contains all newSet? true
After removeAll(): [Apple, Mango, Orange]
After retainAll(): [Apple]
After clear(): [ ]
HashCode: 0
```

## Internal Working of HashSet and LinkedHashSet

1. **Underlying Data Structure**
   - Both use a **hash table** internally to store elements.
   - **LinkedHashSet** adds a **doubly linked list** for maintaining insertion order, but the uniqueness logic is still handled by the hash table.
2. **Uniqueness Check**
   - When an object is added:
     - Its **hashCode()** is computed to determine the **bucket** in the hash table.
     - If the bucket contains objects (hash collision):
       - **equals()** is used to compare objects.
       - If equals() returns **true**, the object is considered a duplicate and **not added**.
       - If equals() returns **false**, the object is added to the bucket.
   - This ensures **no duplicates** exist in either HashSet or LinkedHashSet.
3. **Role of equals() and hashCode()**
   - **hashCode()** → Determines **where the object goes** in the hash table (bucket selection).
   - **equals()** → Determines **if an object is equal to an existing one** in that bucket.

- **Without overriding** these methods for user-defined objects:
  - Objects with the **same data** may be treated as **different**.
  - Sets fail to prevent duplicates.

4. **Order**
   - HashSet: Iteration order is **unpredictable**.
   - LinkedHashSet: Iteration order is **insertion order**, thanks to the linked list, but uniqueness still depends on hashCode() and equals().

## Why We Need to Override equals() and hashCode() for user defined class

### Default Behavior of Object Class

Every class in Java inherits two methods from Object:

1. **equals()**
   - Default: checks **reference equality** (i.e., obj1 == obj2).
   - Means two objects are considered equal **only if they point to the exact same memory location.**
2. **hashCode()**
   - Default: returns a **memory-based hash code.**
   - Two objects with the same content can have **different hash codes.**

So, by default, even if two Student objects have the **same ID and name**, Java considers them **different objects.**

## Contract Between equals() and hashCode()

Whenever you override equals(), you **must override** hashCode().

### Rules:

1. If a.equals(b) is true, then a.hashCode() == b.hashCode() **must be true.**
2. If a.hashCode() == b.hashCode(), it **doesn't guarantee** a.equals(b) is true (hash collision is allowed).
3. Consistency:
   - equals() and hashCode() should consistently return the same result unless object data changes.

## Example with User-Defined Class

**Suppose we have a class Person with fields id and name.**

```
import java.util.Objects;
public class Person {
  private int id;
  private String name;

  // Constructor
  public Person(int id, String name) {
     this.id = id;
     this.name = name;
  }
  // Getters & Setters (optional)
  public int getId() { return id; }
  public String getName() { return name; }

  // 1. Override equals()
  @Override
  public boolean equals(Object obj) {
     if (this == obj) return true; // same reference
     if (obj == null || getClass() != obj.getClass()) return false; // null or different
class
     Person other = (Person) obj;
     return id == other.id && Objects.equals(name, other.name);
  }
  // 2. Override hashCode()
  @Override
  public int hashCode() {
     return Objects.hash(id, name); // generates hash based on fields
  }

  // Optional: for printing
  @Override
  public String toString() {
```

```
        return "Person{id=" + id + ", name='" + name + "'}";
    }
}
```

## Explanation

1. **equals() method**
    - Step 1: this == obj → fast path if same reference.
    - Step 2: obj == null or different class → cannot be equal.
    - Step 3: Cast object and compare **meaningful fields** (here id and name).

2. **hashCode() method**
    - Uses Objects.hash(...) to combine all relevant fields.
    - Ensures that two equal objects have the **same hash code.**

**Using Person with HashSet and LinkedHashSet**

```
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;

public class TestSet {
  public static void main(String[] args) {
    Set<Person> hashSet = new HashSet<>();
    Set<Person> linkedHashSet = new LinkedHashSet<>();

    Person p1 = new Person(1, "Alice");
    Person p2 = new Person(2, "Bob");
    Person p3 = new Person(1, "Alice"); // same as p1

    hashSet.add(p1);
    hashSet.add(p2);
    hashSet.add(p3); // duplicate, will not be added

    linkedHashSet.add(p1);
    linkedHashSet.add(p2);
```

```
        linkedHashSet.add(p3); // duplicate, will not be added

        System.out.println("HashSet: " + hashSet);
        System.out.println("LinkedHashSet: " + linkedHashSet);
    }
}
```

Output:

```
HashSet: [Person{id=1, name='Alice'}, Person{id=2, name='Bob'}]
LinkedHashSet: [Person{id=1, name='Alice'}, Person{id=2, name='Bob'}]
```

****Notice:** p3 is **not added** because it is equal to p1.

**Key Points**

- **Always override equals() and hashCode()** for user-defined objects used in hash-based collections.
- Use **all fields** that define object uniqueness.
- For **LinkedHashSet**, insertion order is maintained automatically.
- If **mutable fields** are used in hashCode() and equals(), changing them after insertion can break set behavior.

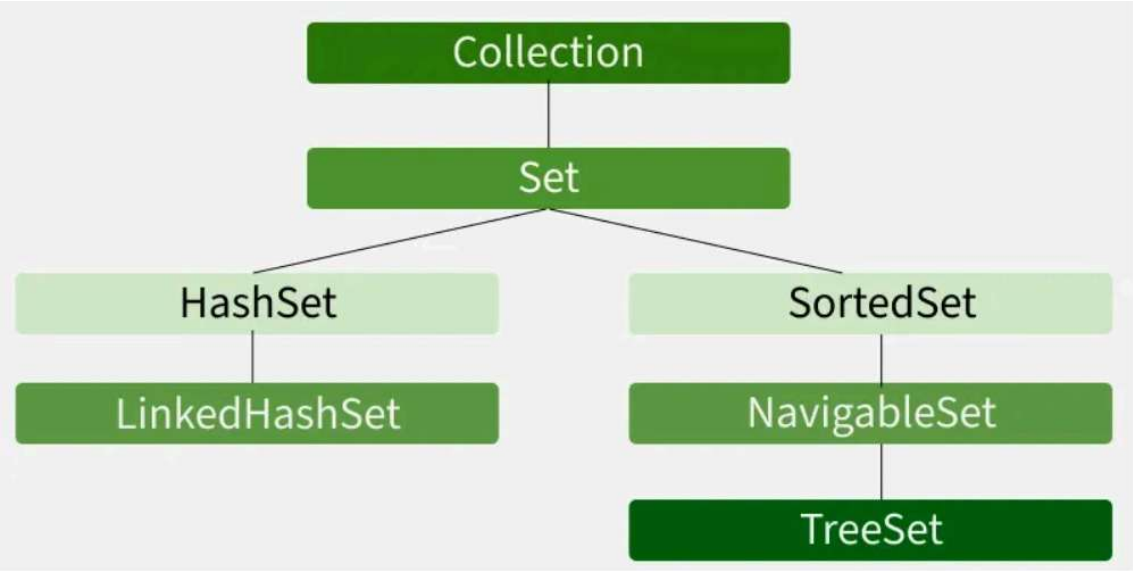
## TreeSet -

TreeSet is a class in Java that:

- Implements the **Set interface**
- Stores **unique elements (no duplicates)**
- Maintains elements in **sorted order (ascending by default)**

## Hierarchy Diagram of TreeSet -

TreeSet implements NavigableSet, which extends SortedSet, which extends Set.




## Package & Syntax

```
import java.util.TreeSet;
TreeSet<Type> set = new TreeSet<>();
```

## Features

- No duplicate elements
- Automatically **sorted (natural ordering)**
- Does **not maintain insertion order**
- Does NOT allow null (throws exception)
- Not synchronized (not thread-safe)

## Internal Working

- Uses a **Red-Black Tree** (self-balancing binary search tree)
- Elements are stored in sorted order
- Each operation maintains tree balance

## So:

- Tree → sorting maintained
- Self-balancing → efficient operations

## Ordering in TreeSet

### Natural Ordering

- Numbers → ascending
- Strings → alphabetical

**Custom Ordering**

- Using Comparator

```
TreeSet<Integer> set = new TreeSet<>((a, b) -> b - a); // descending
```

## Difference: HashSet vs LinkedHashSet vs TreeSet

| Feature | HashSet | LinkedHashSet | TreeSet |
|---|---|---|---|
| Order | No order | Insertion order | Sorted order |
| Structure | Hash table | Hash + Linked List | Red-Black Tree |
| Performance | Fastest | Slightly slower | Slowest |
| Null allowed | Yes | Yes | No |

**Example -**

```
import java.util.*;
public class TreeSet1 {
  public static void main(String[] args) {
    // 1. Creating TreeSet (automatically sorted)
    TreeSet<Integer> set = new TreeSet<>();
    // 2. add() - Adding elements
    set.add(50);
    set.add(10);
    set.add(30);
    set.add(20);
    set.add(40);
    set.add(30); // duplicate ignored
    System.out.println("After adding elements: " + set);
    // 3. size()
    System.out.println("Size: " + set.size());
    // 4. contains()
    System.out.println("Contains 30? " + set.contains(30));
    // 5. remove()
    set.remove(20);
    System.out.println("After removing 20: " + set);
    // 6. isEmpty()
    System.out.println("Is empty? " + set.isEmpty());
```

```
// 7. first() and last()
System.out.println("First (smallest): " + set.first());
System.out.println("Last (largest): " + set.last());
// 8. higher() and lower()
System.out.println("Higher than 30: " + set.higher(30));
System.out.println("Lower than 30: " + set.lower(30));
// 9. ceiling() and floor()
System.out.println("Ceiling of 25: " + set.ceiling(25));
System.out.println("Floor of 25: " + set.floor(25));
// 10. headSet() and tailSet()
System.out.println("HeadSet (<30): " + set.headSet(30));
System.out.println("TailSet (>=30): " + set.tailSet(30));
// 11. subSet()
System.out.println("SubSet (20 to 50): " + set.subSet(20, 50));
// 12. pollFirst() and pollLast()
System.out.println("Poll First: " + set.pollFirst());
System.out.println("Poll Last: " + set.pollLast());
System.out.println("After polling: " + set);
// 13. Iteration using for-each
System.out.println("Using for-each:");
for (int num : set) {
    System.out.println(num);
}
// 14. Iterator
System.out.println("Using Iterator:");
Iterator<Integer> it = set.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
// 15. clone()
TreeSet<Integer> clonedSet = (TreeSet<Integer>) set.clone();
System.out.println("Cloned set: " + clonedSet);
// 16. addAll()
TreeSet<Integer> newSet = new TreeSet<>();
newSet.add(100);
newSet.add(200);
set.addAll(newSet);
```

```
        System.out.println("After addAll(): " + set);
        // 17. containsAll()
        System.out.println("Contains all newSet? " + set.containsAll(newSet));
        // 18. removeAll()
        set.removeAll(newSet);
        System.out.println("After removeAll(): " + set);
        // 19. retainAll()
        TreeSet<Integer> anotherSet = new TreeSet<>();
        anotherSet.add(10);
        anotherSet.add(999);
        set.retainAll(anotherSet);
        System.out.println("After retainAll(): " + set);
        // 20. clear()
        set.clear();
        System.out.println("After clear(): " + set);
        // 21. hashCode()
        System.out.println("HashCode: " + set.hashCode());
    }
}
```

Output -

```
After adding elements: [10, 20, 30, 40, 50]
Size: 5
Contains 30? true
After removing 20: [10, 30, 40, 50]
Is empty? false
First (smallest): 10
Last (largest): 50
Higher than 30: 40
Lower than 30: 10
Ceiling of 25: 30
Floor of 25: 10
HeadSet (<30): [10]
TailSet (>=30): [30, 40, 50]
SubSet (20 to 50): [30, 40]
Poll First: 10
Poll Last: 50
After polling: [30, 40]
```

```
Using for-each:
30
40
Using Iterator:
30
40
Cloned set: [30, 40]
After addAll(): [30, 40, 100, 200]
Contains all newSet? true
After removeAll(): [30, 40]
After retainAll(): [ ]
After clear(): [ ]
HashCode: 0
```

## How TreeSet Checks for Uniqueness and Order

Unlike HashSet and LinkedHashSet, TreeSet **does not use hash codes**. It relies on **comparison** to decide:

1. Where to place an element in the tree (sorting order).
2. Whether an element is a duplicate (uniqueness).

When a new element is added:

- TreeSet uses compareTo() (from Comparable) or compare() (from Comparator) to find its position.
- If compareTo() or compare() returns 0, the element is considered a duplicate and is not added.
- If it returns positive or negative, it is placed accordingly in the tree.

So in TreeSet, comparison defines both ordering and uniqueness.

### Why Comparable is Needed

- Comparable is an interface for natural ordering of objects.
- It has one method:
  int compareTo(T o)

**Example**: A Student class may define natural order by rollNumber:

```
public int compareTo(Student other) {
   return this.rollNumber - other.rollNumber;}
```

### Benefits:

- TreeSet knows how to sort students automatically.
- TreeSet knows how to detect duplicates (if compareTo() returns 0).

## Why is a Comparator Needed

- Comparator is an interface for custom ordering.
- It has one method:
  `int compare(T o1, T o2)`

### Use cases:

- If you cannot modify the class to implement Comparable.
- If you want multiple sorting strategies (e.g., by name, by marks, by roll number).

**Example**:

```
Comparator<Student> nameComparator = new Comparator<Student>() {
   public int compare(Student s1, Student s2) {
      return s1.getName().compareTo(s2.getName());
   }
};
```

**Benefits**:

- Allows flexible, custom sorting without changing the class.
- Still ensures uniqueness (if compare returns 0, the TreeSet treats as a duplicate).

## Comparable interface in Java:

In Java, Comparable is an interface that belongs to **java.lang** package. which has only one method.

```
public int compareTo(Object obj);
```

In order to add our User-defined Java classes object inside the TreeSet, we should implement the Comparable interface inside our User-defined class and need to specify the sorting logic by overriding the compareTo method.

The compareTo method is used to compare the current object with the specified object. **It returns:**

- positive integer, if the current object is greater than the specified object.
- negative integer, if the current object is less than the specified object.
- zero, if the current object is equal to the specified object.

**Note:** All the Wrapper classes and the String class internally implement the Comparable interface.

**Example:** implementing the Comparable interface inside the Student class. According to the roll number.

```
package com.masai;
public class Student implements Comparable<Student>{

    private int roll;
    private String name;
    private int marks;

    public Student() {
    }

    public Student(int roll, String name, int marks) {
        this.roll = roll;
        this.name = name;
        this.marks = marks;
    }

    public int getRoll() {
        return roll;
    }

    public void setRoll(int roll) {
        this.roll = roll;
    }

    public String getName() {
        return name;
    }
```

```
    public void setName(String name) {
        this.name = name;
    }

    public int getMarks() {
        return marks;
    }

    public void setMarks(int marks) {
        this.marks = marks;
    }

    @Override
    public String toString() {
        return "Student{" +
                "roll=" + roll +
                ", name='" + name + '\\' +
                ", marks=" + marks +
                '}';
    }

    @Override
    public int compareTo(Student student) {

        if(this.roll > student.roll)
            return +1;
        else if(this.roll < student.roll)
            return -1;
        else
            return 0;

    }
}
```

Now we can add the Student object inside the TreeSet object.

**Example:**

```
import java.util.TreeSet;
class Main{
   public static void main(String args[]){

      TreeSet<Student> ts = new TreeSet<>();

      ts.add(new Student(20,"Amit",520));
      ts.add(new Student(15,"Suresh",550));
      ts.add(new Student(22,"Ajay",540));
      ts.add(new Student(18,"Rajesh",590));

      System.out.println(ts);
   }
}
```

Overriding the compareTo method to sort the object according to the name.

```
@Override
public int compareTo(Student student) {
   return name.compareTo(student.name);
}
```

## Comparator interface in Java:

If we want to define the sorting logic for objects of some class, outside that class, then we can use the Comparator interface.

This Comparator interface belongs to **java.util** package. This interface also has only an abstract method :

```
public int compare(Object obj1, Object obj2);
```

Let's define the sorting logic of the Student object outside the Student class, i.e., inside another class using the Comparator interface.

```
//StudentRollComparator.java
import java.util.Comparator;

public class StudentRollComparator implements Comparator<Student> {
```

```
    @Override
    public int compare(Student s1, Student s2) {
        if(s1.getRoll() > s1.getRoll())
            return +1;
        else if(s1.getRoll() < s2.getRoll())
            return -1;
        else
            return 0;
    }
}
```

Now we can add the Student object inside the TreeSet collection object by mentioning this “StudentRollComparator” class in the constructor of the TreeSet object.

In this case Student object need not implement the Comparable interface anymore.

**Example:**

```
import java.util.TreeSet;
class Main{
    public static void main(String args[]){
        **TreeSet<Student> ts = new TreeSet<>(new
StudentRollComparator());**

        ts.add(new Student(20,"Amit",520));
        ts.add(new Student(15,"Suresh",550));
        ts.add(new Student(22,"Ajay",540));
        ts.add(new Student(18,"Rajesh",590));

        System.out.println(ts);
    }
}
```

## Difference between Comparable and Comparator:

| Comparable | Comparator |
|---|---|
| Comparable interface belongs to java.lang package | The Comparator interface belongs to java.util package |
| If we want to specify the sorting logic of a class object within the same class, we need to use Comparable | If we want to specify the sorting logic of a class object outside that class, then we should use Comparator. |
| With the help of Comparable, we can define only one sorting logic within a class. | With the help of Comparator, we can define multiple sorting logics of a class object inside multiple classes. |
| The method name here is: public int compareTo(Object obj) | Here method name is : public int compare(Object obj1, Object obj2) |

# Java Collections Class

## Introduction

- The Collections class in Java is part of java.util package.
- It contains static utility methods for operations on Collection objects (like List, Set, etc.).
- It does not implement any collection itself, but provides methods for:
    1. Sorting
    2. Searching
    3. Shuffling
    4. Finding min/max
    5. Synchronization
    6. Filling, copying, and reversing collections

Think of Collections as a toolbox for collections.

## Key Points

1. Works only with Collection framework classes (List, Set, Queue) and their implementations.
2. All methods are static.
3. Methods can work on any Collection type, but some are specific to List (like sort(), reverse()).

# Commonly Used Methods and Examples

## 1. Sorting

```
import java.util.*;
public class CollectionsExample {
  public static void main(String[] args) {
    List<Integer> numbers = new ArrayList<>(Arrays.asList(5, 3, 8, 1, 2));
    // Sort in ascending order
    Collections.sort(numbers);
    System.out.println("Ascending: " + numbers);


    // Sort in descending order
    Collections.sort(numbers, Collections.reverseOrder());
    System.out.println("Descending: " + numbers);
  }
}
Output:
Ascending: [1, 2, 3, 5, 8]
Descending: [8, 5, 3, 2, 1]
```

## 2. Reverse

```
List<String> names = new ArrayList<>(Arrays.asList("Alice", "Bob",
"Charlie"));
Collections.reverse(names);
System.out.println("Reversed: " + names);
Output:
Reversed: [Charlie, Bob, Alice]
```

### 3. Shuffle

```
List<Integer> cards = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
Collections.shuffle(cards);
System.out.println("Shuffled: " + cards);
```

**Randomizes order, useful for games or random sampling.**

### 4. Max and Min

```
List<Integer> nums = Arrays.asList(10, 20, 5, 40, 30);
System.out.println("Max: " + Collections.max(nums));
System.out.println("Min: " + Collections.min(nums));
Output:
Max: 40
Min: 5
```

### 5. Frequency

```
List<String> list = Arrays.asList("apple", "banana", "apple", "orange", "apple");
System.out.println("Frequency of apple: " + Collections.frequency(list,
"apple"));
Output:
Frequency of apple: 3
```

### 6 Copy

```
List<String> source = Arrays.asList("A", "B", "C");
List<String> dest = new ArrayList<>(Arrays.asList("", "", ""));
Collections.copy(dest, source);
System.out.println("Destination: " + dest);
Output:
Destination: [A, B, C]
```

**Note: Destination list must be at least as big as the source list.**

### 7. Fill

```
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4));
Collections.fill(list, 0); // Fill with 0
System.out.println("Filled List: " + list);
```

Output:

Filled List: [0, 0, 0, 0]

### 8. Replace All

```
List<String> list = new ArrayList<>(Arrays.asList("Java", "Python", "Java"));
Collections.replaceAll(list, "Java", "C++");
System.out.println(list);
```

Output:

[C++, Python, C++]

### 9. Reverse Order (Comparator)

```
List<Integer> nums = Arrays.asList(1, 4, 2, 5);
nums.sort(Collections.reverseOrder());
System.out.println(nums);
```

Output:

[5, 4, 2, 1]

## Map Interface in Java

Using Map, we can also group multiple objects in the form of a key-value pair. here each key-value pair is known as an entry.

A Map is useful if we want to search, update, or delete elements on the basis of a key.

**Example:**

- A map of error codes and their descriptions.
- A map of zip codes and cities.
- A map of managers and employees. Each manager (key) is associated with a list of employees (value) he manages.
- A map of classes and students. Each class (key) is associated with a list of students (value).

**Map Interface:**

- It is not a child interface of the Collection interface.
- This interface belongs to **java.util** package.
- The map is able to arrange all the elements in the form of key-value pairs.
- In Map, both keys and values are objects.

Declaration of the Map interface

```
public interface Map<K, V>
```

- K -> Type of keys maintained by the map
- V -> Type of mapped values

## Some of the Important Methods of Map:

| Method | Description |
|---|---|
| public V put(K key, V value) | Associates the specified value with the specified key in this map. It is used to insert an entry in the map. |
| public V get(Object key) | This method returns the object that contains the value associated with the key. |
| public V remove(Object key) | It is used to delete an entry for the specified key. |
| public void clear() | It is used to reset the map. |
| public int size() | This method returns the number of entries in the map. |
| public boolean containsKey(Object key) | Returns true if this map contains a mapping for the specified key. |
| public boolean containsValue(Object value); | Returns true if this map has one or more keys to the specified value. |

| | |
|---|---|
| public Set<K> keySet() | Returning a **Set** view of the keys contained in this map. |
| public Collection<V> values() | Returns a **Collection** view of the values contained in this map. |
| public Set<Map.Entry<K,V>> entrySet() | Returns a **Set**view of the mappings contained in this map(set of Entry). |
| public void putAll(Map map) | Copies all of the mappings from the specified map to this map |

## Map.Entry Interface:

Entry is the inner interface defined inside the Map interface. So we will be accessed it by Map.Entry name. It returns a collection-view of the map.

Without existing Map object there is no-chance of existing entry object(because of that Entry interface is defined inside Map interface).

Each key value pair is called as one entry ,hence map is considered as a group of entries.

**Methods of Map.Entry interface:**

- **public K getKey()**; It is used to obtain a key.
- **public V getValue()**; It is used to obtain a value.
- **public V setValue(V value)**; It is used to replace the value corresponding to this entry with the specified value.

**Note**: Since Entry is an inner interface defined inside the Map interface, we define the variable of this Entry interface with the help of its outer interface Map.

**Example**:

Map.Entry me;

Implemented Classes of the Map Interface

- **HashMap:** Stores key-value pairs using hashing for fast access, insertion, and deletion.
- **LinkedHashMap:** Similar to HashMap, but maintains the insertion order of key-value pairs.
- **TreeMap:** Stores key-value pairs in sorted order using natural ordering or a custom comparator.
- **Hashtable:** A synchronized Map implementation that doesn't allow null keys or values.

## HashMap class:

The Java **HashMap** class implements the Map interface, which allows us *to store key and value pairs*, where keys should be unique. If you try to insert the duplicate key, it will replace the value of the corresponding key.

Elements will be inserted based on the hash code of the "key", so it does not follow the sequence.

We can add only one null value as a key, but any number of null values can be added as a value.

**Example**:

```
import java.util.HashMap;
public class Main {
   public static void main(String[] args) {

      HashMap<Integer,String> hm = new HashMap<>();
      System.out.println(hm);//{}
      hm.put(1,"one");
      hm.put(2,"two");
      hm.put(3,"three");

      System.out.println(hm);

      hm.put(null,"four"); //one null is allowd as a key
            hm.put(null,"FOUR"); //four will be replaced with FOUR
      hm.put(2,"TWO"); // two will be replaced with TWO
      hm.put(5,null);
      hm.put(6,null); //as a value any number of null be allowed

      System.out.println(hm.size());
      System.out.println(hm);

   }
}
```

```
Output:
{}
{1=one, 2=two, 3=three}
6
{null=FOUR, 1=one, 2=TWO, 3=three, 5=null, 6=null}
```

**Example 2: Iterating over a Map:**

```
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.Set;

public class Main {

   public static void main(String[] args) {

      Map<Integer,String> hm = new HashMap<>();
      hm.put(1,"one");
      hm.put(2,"two");
      hm.put(3,"three");
      hm.put(4,"four");
      hm.put(5,"five");

      System.out.println("Getting all the keys");
      Set<Integer> keys = hm.keySet();
      for(Integer x:keys){
         System.out.println(x);
      }

      System.out.println("=================");

      System.out.println("Getting all the values");
      Collection<String> values= hm.values();
      for(String value:values){
         System.out.println(value);
```

```
        }
        System.out.println("=================");

        System.out.println("Getting both key and values");

        Set<Map.Entry<Integer,String>>  keyValue= hm.entrySet();

        for(Map.Entry<Integer,String> me: keyValue){

            System.out.println(me.getKey()+"======"+me.getValue());
        }
    }
}
```

**Example:**

Let's create a HashMap for the State topper students, and print them back.

```
//Student.java
public class Student  {

    private int roll;
    private String name;
    private int mark;

    public Student() {
    }

    public Student(int roll, String name, int mark) {
        this.roll = roll;
        this.name = name;
        this.mark = mark;
    }

    public int getRoll() {
        return roll;
```

```
    }

    public void setRoll(int roll) {
        this.roll = roll;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getMark() {
        return mark;
    }

    public void setMark(int mark) {
        this.mark = mark;
    }
}

//Main.java
import java.util.*;

public class Main {

    public static void main(String[] args) {

        HashMap<String,Student> hm = new HashMap<>();

        hm.put("Maharastra",new Student(10,"Ganesh",950));
        hm.put("Tamilnadu",new Student(12,"Surya",850));
        hm.put("Telangana",new Student(15,"Venkat",920));
        hm.put("Haryana",new Student(16,"Dinesh",910));
        hm.put("Kerla",new Student(18,"Srinu",880));
```

```
        //getting all the state names:
        Set<String> states= hm.keySet();

        for(String state:states){
            System.out.println(state);
        }

        //Getting all the students as a List object.
        Collection<Student> cols =  hm.values();
        List<Student> students = new ArrayList<>(cols);

        //printing all the students from the List
        for(Student student:students){
            System.out.println("Roll :"+student.getRoll());
            System.out.println("Name :"+student.getName());
            System.out.println("Marks :"+student.getMark());
            System.out.println("==========================");
        }

        //printing state-wise students' details:

        Set<Map.Entry<String,Student>> set = hm.entrySet();

        for(Map.Entry<String,Student> me: set) {
            System.out.println("Toppers Student of State :" + me.getKey());
            System.out.println("**********************************");

            Student student = me.getValue();

            System.out.println("Roll :" + student.getRoll());
            System.out.println("Name :" + student.getName());
            System.out.println("Marks :" + student.getMark());
        }
    }
}
```

# How HashMap Works Internally

## Step 1: Compute Hash

- When you put a key-value pair: map.put(key, value)
- Java calls key.hashCode() to get an **integer hash code**.
- The hash code is then processed to determine the **bucket index**:

  index = hash(key.hashCode()) % capacity

- The hash() function improves **distribution** to reduce collisions.

## Step 2: Locate Bucket

- The computed index points to a **bucket in the internal array**.
- Each bucket can store **multiple entries** in a linked list or tree (collision handling).

## Step 3: Check for Key Equality

- If the bucket is **empty**, the new key-value pair is inserted directly.
- If the bucket has **existing entries**:
  1. Iterate over entries in the bucket.
  2. Compare the keys using **equals()**.
  3. If a key matches (equals() == true):
     - **Update the value** for that key.
  4. If no match is found:
     - **Add a new entry** in the bucket (linked list or tree).

  So hashCode() → selects bucket, equals() → checks for duplicate key in that bucket.

## Step 4: Handle Collisions

- Multiple keys may produce the **same bucket index** (hash collision).

- Java handles this with:
    - **Linked list** for <= 8 entries.
    - **Red-Black Tree** for > 8 entries (improves performance from O(n) to O(log n)).

### Step 5: Resize

- When the load factor exceeds 0.75 (default), the HashMap **doubles its capacity**.
- All existing entries are **rehashed** to the new buckets.

## LinkedHashMap class:

It is a child class of HashMap, and it will preserve the sequence of all the entries.

**Example**:

```
import java.util.*;
class Main{
 public static void main(String args[]){

  LinkedHashMap<Integer,String> hm=new LinkedHashMap<>();

  hm.put(100,"Amit");
  hm.put(101,"Vijay");
  hm.put(102,"Rahul");

for(Map.Entry<Integer,String> me:hm.entrySet()){
   System.out.println(me.getKey()+" "+me.getValue());
     }
    }
}
```

## TreeMap class:

TreeMap class implements the **Map** and **SortedSet** interface; here map will be sorted according to the natural ordering of its keys, or by a **Comparator** provided at map creation time, depending on which constructor is used.

If we don't use Comparator, then the key object must be **Comparable.** Otherwise, it will raise a ClassCastException.

The TreeMap in Java does not allow null keys (like TreeSet), and thus, a **NullPointerException** is thrown. However, multiple null values can be associated with different keys.

**Example**:

```
public class Main {
   public static void main(String[] args) {

      TreeMap<String,String> tm = new TreeMap<>();
      tm.put("Maharastra","Mumbai");
      tm.put("Telangana","Hydrabad");
      tm.put("Tamilnadu","Chennai");
      tm.put("Karnataka","Bangaluru");
      tm.put("Bihar","Patna");

      System.out.println(tm);
   }
}
```

Output:

{Bihar=Patna, Karnataka=Bangaluru, Maharastra=Mumbai, Tamilnadu=Chennai, Telangana=Hydrabad}

## TreeMap class using Comparator:

The Key of the TreeMap should be Comparable; if the key is not Comparable, then we need to use the Comparator and specify the class that implements the Comparator to the TreeMap constructor.

**Example**: Student as a key and state as a value.

```
//StudentMarksComp.java
import java.util.Comparator;
public class StudentMarksComp implements Comparator<Student> {
   @Override
```

```
    public int compare(Student s1, Student s2) {
        if(s1.getMark() > s2.getMark())
            return +1;
        else if(s1.getMark() < s2.getMark())
            return -1;
        else
            return 0;
    }
}

//Main.java
import java.util.*;
public class Main {
    public static void main(String[] args) {

        TreeMap<Student,String> tm = new TreeMap<>(new
StudentMarksComp());

        tm.put(new Student(10,"Ganesh",950),"Maharastra");
        tm.put(new Student(12,"Surya",850),"Tamilnadu");
        tm.put(new Student(15,"Venkat",920),"Telangana");
        tm.put(new Student(16,"Dinesh",910),"Haryana");
        tm.put(new Student(18,"Srinu",880),"Kerla");

        Set<Map.Entry<Student,String>> set= tm.entrySet();

        for(Map.Entry<Student,String> me: set) {
            System.out.println("Toppers Student of State :" + me.getValue());
            System.out.println("**********************************");

            Student student = me.getKey();

            System.out.println("Roll :" + student.getRoll());
            System.out.println("Name :" + student.getName());
            System.out.println("Marks :" + student.getMark());
        }
    }
```

```
}
```

## Java Hashtable class:

HashMap and Hashtable both are used to store data in key and value form. Both are using the hashing technique to store unique keys.

But there are the following main differences between the HashMap and the Hashtable classes:

1. HashMap is **non-synchronized**. It is not thread-safe and cannot be shared between many threads without proper synchronization code. whereas Hashtable is **synchronized.** It is thread-safe and can be shared with many threads.
2. HashMap **allows one null key and multiple null values,** whereas Hashtable **doesn't allow any null key or value.**

Example: ****

```
import java.util.*;
class Main{
 public static void main(String args[]){

 Hashtable<Integer,String> hm=new Hashtable<Integer,String>();

  hm.put(100,"Amit");
  hm.put(102,"Ravi");
  hm.put(101,"Vijay");
  hm.put(103,"Rahul");

  for(Map.Entry<Integer,String> m:hm.entrySet()){
   System.out.println(m.getKey()+" "+m.getValue());
  }
 }
}
```

# GENERICS IN JAVA

## Introduction to Generics

Generics in Java were introduced in Java 5 to allow the creation of classes, interfaces, and methods that can operate on any defined type while providing compile-time type safety. The key idea behind generics is to enable code reusability and flexibility without compromising type safety.

**Think of Generics as:**

Type Placeholders

<T> <E> <K,V>

**Instead of writing:**

StringBox

IntegerBox

EmployeeBox

**We write:**

Box<T>

**And use:**

Box<String>

Box<Integer>

Box<Employee>

## Why Generics? (Problem Without Generics)

Before Java 5, everything was handled using the **Object class**.

**Without Generics (Pre-Java 5)**

```
class MyGen {
   private Object obj;
   void add(Object obj) {
      this.obj = obj;
   }
   Object get() {
```

```
        return obj;
    }
}
```

### Usage:

```
MyGen gen = new MyGen();
gen.add("Hello");
Object obj = gen.get();
String s = (String) obj; // Manual casting required
```

## Problems Without Generics

1. No Type Safety
2. Manual Casting Required
3. Runtime Errors Possible
4. ClassCastException Risk

## With Generics (Solution)

```
class MyGen<T> {
    private T obj;
    void add(T obj) {
        this.obj = obj;
    }
    T get() {
        return obj;
    }
}
```

### Usage:

```
MyGen<String> gen = new MyGen<>();
gen.add("Hello");
String s = gen.get(); // No casting
```

## Type Safety — Core Benefit

**Type Safety in Java:** Type safety means you can only store and use the correct type of data, and errors are caught at compile time, not runtime.

## Without Generics (Unsafe)

```
import java.util.ArrayList;
public class Test {
  public static void main(String[] args) {
     ArrayList list = new ArrayList(); // raw type
     list.add("Vivek");
     list.add(100); // allowed
     String name = (String) list.get(1); // Runtime error
     System.out.println(name);
  }
}
```

Compiler allows mixed types

At runtime → ClassCastException  Problem: No type safety

## With Generics (Safe)

```
import java.util.ArrayList;
public class Test {
  public static void main(String[] args) {
     ArrayList list = new ArrayList(); // raw type
     list.add("Vivek");
     list.add(100); // allowed
     String name = (String) list.get(1); // Runtime error
     System.out.println(name);
  }
}
```

Compiler allows mixed types

At runtime → ClassCastException  Problem: No type safety

## Key Benefit

- Only String allowed
- Error caught at **compile time**

- No runtime crash
- This is **Type Safety**

Type safety ensures that only the intended data type is used, preventing runtime errors by catching issues at compile time.

## Important Rule — Reference Types Only

Generics **work only with reference types**.

**Invalid:**

```
List<int> list;
```

**Valid:**

```
List<Integer> list;
```

Use Wrapper Classes:

| Primitive | Wrapper |
|---|---|
| int | Integer |
| double | Double |
| char | Character |

# Generic Class

A **generic class** uses a type parameter <T>.

```
class Box<T> {
  private T value;
  public void set(T value) {
     this.value = value;
  }
  public T get() {
     return value;
  }
 public static void main(String[] args) {
```

```
Box<Integer> intBox = new Box<>();
intBox.set(100);
Box<String> strBox = new Box<>();
strBox.set("Hello");
}
}
```

T = Type Parameter (can be any type);

### Diagram Logic

```
Box<T>{}
```

T replaced by:

```
Box<Integer>{}
Box<String>{}
```

## Generic Methods

A generic method is a method that can work with any data type, using a type parameter <T>.

### Basic Syntax:

```
public <T> void print(T data) {
  System.out.println(data);
}
<T> comes before return type T can be any type (String, Integer, etc.)

Example
class Test {
  public static <T> void show(T value) {
    System.out.println(value);
  }
  public static void main(String[] args) {
    show("Hello");  // String
    show(100);      // Integer
```

```
        show(10.5);     // Double
    }
}
```

**O/P:-**

```
Hello
100
10.5
```

**Same method works for multiple types → Reusability + Type Safety**

## Generic Method with Return Type:

```
class Test {
    public static <T> T getData(T value) {
        return value;
    }
    public static void main(String[ ] args) {
        int x = getData(10);
        String s = getData("Vivek");
        System.out.println(x);
        System.out.println(s);
    }
}
```

## Multiple Type Parameters

```
class Test {
    public static <K, V> void print(K key, V value) {
        System.out.println(key + " : " + value);
    }
    public static void main(String[] args) {
        print(1, "Java");
        print("Age", 25);
    }
}
```

# Bounded Type Parameters

In Generic Classes, we can apply a constraint or bound the type parameter for a particular range by using the "extends" keyword.

**Example:**

```
class Test<T> {
    T value;
}
Test<String>
Test<Integer>
Test<Object>
```

Imagine you want a method that **only works with numbers** because you want to do math like square, addition, etc.

```
class Test<T> {
  T value;
  void show() {
     // System.out.println(value * value);  Not possible
  }
}
```

Compiler doesn't know what T is; it could be String, Boolean, anything.

## With Bounded Type :

```
class Test<T extends Number> {
  T value;
  void show() {
    // System.out.println(value * value);
  }
}
// Now Java knows T will always be a Number.
public class Main {
  public static void main(String[] args) {
     Test<Integer> t1 = new Test<>();
     t1.value = 5;
     t1.show();
     Test<Double> t2 = new Test<>();
```

```
        t2.value = 2.5;
        t2.show();

        Test<String> t3 = new Test<>();//Compile-time error, String is NOT a
    Number
      }
    }
```

### Real-Life Example

```
    class Payment { }
    class CashPayment extends Payment { }
    class CardPayment extends Payment { }
    class Bill<T extends Payment> { }

    Valid:
    Bill<CashPayment>
    Bill<CardPayment>
    Bill<Payment>

    Invalid:
    Bill<String>
```

## Multiple Bounds

-In Generic classes, we can use more than one type as a bounded parameter by using the '&' symbol.

**Example**

```
    class Test<T extends Number & Serializable>
    {
    }
```

Here, the Test class is able to allow the elements that must be either the same as Number or subclasses of the number and must be implementations of the Serializable interface.

```
    Test<Integer> t=new Test<Integer>();--> Valid
```

Test<Float> t=new Test<Float>();--> Valid

Test<String> t=new Test<String>--> Invalid.(Because String type is not a subtype of Number, but its implements the Serializable interface)

## Rule:

- Class first
- Interface second

The "extends" is able to allow both class type and interface type, but first we have to specify the class type, then we have to specify the interface type. i.e., first class name and then interface name.

## Example:-
## Valid Syntax:-

```
class Test<T extends Number & Runnable>
{
}
```

## Invalid Syntax:-

```
class Test<T extends Runnable & Number>
{
}
```

Note:- The "extends" keyword is able to allow more than one interface type.

```
class Test<T extends Runnable & Serializable>
{
}
```

## Example:-

```
class Employee {

    private String name;
    private double salary;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
```

```
    public double getSalary() {
        return salary;
    }

    public String getName() {
        return name;
    }
}
interface BonusEligible {
    double calculateBonus();
}
class Developer extends Employee
        implements BonusEligible, Serializable {

    public Developer(String name, double salary) {
        super(name, salary);
    }

    @Override
    public double calculateBonus() {
        return getSalary() * 0.10;
    }
}
//Generic Class with Multiple Bounds
class Payroll<T extends Employee
                & BonusEligible
                & Serializable> {

    private T employee;

    public Payroll(T employee) {
        this.employee = employee;
    }

    public void processSalary() {
```

```
        System.out.println(
                "Employee: " + employee.getName());

        System.out.println(
                "Salary: " + employee.getSalary());

        System.out.println(
                "Bonus: " + employee.calculateBonus());
    }
}

public class Main {

    public static void main(String[] args) {

        Developer dev =
                new Developer("Rahul", 50000);

        Payroll<Developer> payroll =
                new Payroll<>(dev);

        payroll.processSalary();
    }
}
```

Output:

—--------

Employee: Rahul

Salary: 50000.0

Bonus: 5000.0

**NOTE:**

- Bounded parameters or constraints at the class level do not allow the "implements" keyword and the "super" keyword.
- The "extends" keyword will not allow two class types at a time.
  Invalid Syntax:-

```
class Test<T extends Number & Thread>
{
}
```

## Covariance type in Java:-

-As we know, to the super class reference variable we can store any of its subclass objects.

### Example:

```
String s = "Hello";
Object obj = s;
```

-Covariance allows a type to accept its subtypes while preserving the "is-a" relationship.

-In Java, arrays are the covariance types:

-We can store an array of any object in the reference variable of Object[] array also.

### Example:

```
String[] fruits = {"Apple", "Mango", "Banana", "Pineapple"};
Object[] objArray = fruits; //allowed
```

### Problem:

```
objArray[0] = new A(); // Heap Pollution, ArrayStoreException
```

Here, no error at compile time; the compiler is not able to detect it, but at runtime, we get an exception.

## Note:-

**Generics are invariant in Java, meaning there is no subtype relationship between parameterized types, even if their base types are related.**

**Example 1:**

```
List<String> sList = new ArrayList<>();
List<Object> oList = sList; // Compilation error
```

**Example 2:**

```
public static void printArray(List<Employee> employees){

   for(Employee employee: employees){
      System.out.println(employee);
   }

}
List<Employee> employees = List.of(new Employee(), new Employee());
printArray(employees); // valid

List<Developer> developers = List.of(new Developer(), new Developer());
printArray(developers); // Error
```

## Here is one way:

```
List<Developer> delevopers = new ArrayList<>(List.of(new Developer(), new Developer()));

List<Employee> employees = new ArrayList<>();

for(Developer developer: delevopers) {

   employees.add(developer);
}
printArray(employees);
```

- To make our Collection work as a covarience type and make them flexible, we need to make use of wildcards.

# Wildcards in Generics

In Java Generics, wildcards are used when you don't know the exact type. They let you write flexible and reusable code. The wildcard is represented by a ? (question mark). Wildcards are mainly used in method parameters to accept different generic types safely.

## Types of wildcards in Java

### 1. Upper Bounded Wildcards (Covariance )(for reading/producing)

Using this we can achieve the covariance type of feature in collections also.

These wildcards can be used when you want to relax the restrictions on a variable. For example, say you want to write a method that works on List < Integer >, List < Double >, and List < Number >, you can do this using an upper-bounded wildcard.

To declare an upper-bounded wildcard, use the wildcard character ('?'), followed by the extends keyword, followed by its upper bound.

*public static void add(List<? extends Number> list)*

### Implementation:

```
import java.util.Arrays;
import java.util.List;

class WildcardDemo {
  public static void main(String[ ] args)
  {
     // Upper Bounded Integer List
     List<Integer> list1 = Arrays.asList(4, 5, 6, 7);

     System.out.println("Total sum is:" + sum(list1));

     // Double list
     List<Double> list2 = Arrays.asList(4.1, 5.1, 6.1);
```

```
        System.out.print("Total sum is:" + sum(list2));
    }

    private static double sum(List<? extends Number> list)
    {
        double sum = 0.0;
        for (Number i : list) {
            sum += i.doubleValue();
        }

        return sum;
    }
}
```

Output

```
Total sum is:22.0
Total sum is:15.299999999999999
```

**Explanation:** In the above program, list1 holds Integer values and list2 holds Double values. Both are passed to the sum method, which uses a wildcard <? extends Number>. This means it can accept any list of a type that is a subclass of Number, like Integer or Double.

**Note**: When we are using Subtype or Upper bound Wildcard, we can't add a new element/object inside the collection.

## 2. Lower Bounded Wildcards: Contravariance (for writing/consuming)

Contravariance is the opposite of covariance, and it refers to the ability of a type to accept more generic(broader) types.

It is expressed using the wildcard character ('?'), followed by the super keyword, followed by its lower bound: <? super T>.

Lower bounded wildcard (? super T) allows a generic type to accept T and its supertypes, making it suitable for write operations.

***Syntax:***

***Collectiontype <? super A>***

**Implementation:**

```
import java.util.Arrays;
import java.util.List;

class WildcardDemo {
  public static void main(String[] args)
  {
     // Lower Bounded Integer List
     List<Integer> list1 = Arrays.asList(4, 5, 6, 7);

     // Integer list object is being passed
     printOnlyIntegerClassorSuperClass(list1);

     // Number list
     List<Number> list2 = Arrays.asList(4, 5, 6, 7);

     // Integer list object is being passed
     printOnlyIntegerClassorSuperClass(list2);
  }

  public static void printOnlyIntegerClassorSuperClass(
     List<? super Integer> list)
  {
     list.add(10);
     list.add(20);
     System.out.println(list);
  }
}
Output
[4, 5, 6, 7, 10, 20]
[4, 5, 6, 7, 10, 20]
```

**Note:**

**1. Write (Allowed)**

list.add(10); // Integer allowed, Because all these lists can store Integer

**2. Read (Restricted):**

Object obj = list.get(0); // Only Object allowed, because Compiler doesn't know exact type

**Explanation:** Here, the method printOnlyIntegerClassorSuperClass accepts only Integer or its superclasses (like Number). If you try to pass a list of Double, it gives a compile-time error because Double is not a superclass of Integer.

***Note:*** *Use extend wildcard when you want to get values out of a structure and super wildcard when you put values in a structure. Don't use wildcards when you get and put values in a structure. You can specify an upper bound for a wildcard or you can specify a lower bound, but you cannot specify both.*

***Note****: We can specify an upper bound for a wildcard, or we can specify a lower bound but we can not specify both.*

## 3. Unbounded Wildcard

This wildcard type is specified using the wildcard character (?), for example, List. This is called a list of unknown types. These are useful in the following cases -

- When writing a method that can be employed using the functionality provided in the Object class.
- When the code is using methods in the generic class that don't depend on the type parameter

**Implementation:**

```
import java.util.Arrays;
import java.util.List;

class unboundedwildcardemo {
  public static void main(String[] args)
  {

    // Integer List
    List<Integer> list1 = Arrays.asList(1, 2, 3);
```

```
        // Double list
        List<Double> list2 = Arrays.asList(1.1, 2.2, 3.3);

        printlist(list1);

        printlist(list2);
    }
    private static void printlist(List<?> list)
    {
        System.out.println(list);
    }
}
```

**Output**

```
[1, 2, 3]

[1.1, 2.2, 3.3]
```

**PECS Principle:-**
Producer → extends
Consumer → super

**Remember:**

**PECS**

- Producer Extends
- Consumer Super

## Covariance vs Invariance

| Feature | Arrays | Generics |
|---|---|---|
| Behavior | Covariant | Invariant |
| Safety | Runtime | Compile-time |

# Type Erasure in Java

Type Erasure is the process by which the Java compiler removes all generic type information and replaces type parameters with their upper bounds (or Object if unbounded). **Type Erasure** is a core concept in Java Generics that explains *how generics are implemented internally*.

During compilation, the compiler performs:

- Replacing type parameters with their upper bound or Object.
- Inserting casts where needed to maintain type safety.
- Generating bridge methods to preserve method overriding in generic hierarchies.

***Note:*** *After compilation, no generic type exists in the bytecode. Only raw types remain.*

When you write generics like:

```
List<String> list = new ArrayList<>();
```

At **compile time**, Java uses String for type safety
But at **runtime**, Java **removes (erases)** the generic type

**So it becomes:**

```
List list = new ArrayList();
```

This process is called **Type Erasure.**

**Why Java Uses Type Erasure -**

Java introduced generics in Java 5, but it needed to:

- Stay compatible with older code (backward compatibility)
- Avoid changing the JVM

**So instead of adding new runtime behavior, Java:**
- Checks types at compile time
- Removes them at runtime

## How Type Erasure Works

**1. Replace Generic Types with Object (or Bound)**

**Example 1:**

```
class Box<T> {

  T value;
```

```
}
```

**After type erasure:**

```
class Box {
  Object value;
}
```

**2. If Bounded → Replace with Bound**

```
class Box<T extends Number> {
  T value;
}
```

**After type erasure:**

```
class Box {
  Number value;
}
```

**3. Add Type Casting Automatically**

```
Box<String> box = new Box<>();
String s = box.value;
```

**After type erasure:**

```
Box box = new Box();
String s = (String) box.value; // compiler adds this cast
```

## Type Erasure in Action

Now, we will see examples to understand how type erasure affects code during runtime.

**Example 1: Using Generics**

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Demo{

  public static void main(String args[]){

    List<String> list = new ArrayList<>();
    list.add("Hello");

    Iterator<String> iter = list.iterator();
    while (iter.hasNext()) {
      String s = iter.next();
      System.out.println(s);
    }
  }
}
Output
Hello
```

**Explanation:**

- Compiler checks type safety -> OK (String list)
- At runtime, the generic type (String) does not exist.
- Type casts inserted by the compiler ensure safety.

**Example 2: Raw Types**

```
import java.util.*;
public class Demo{
   public static void main(String args[]){
     List list = new ArrayList();   // Raw type
     list.add("Hello");
     String s;
     for (Iterator iter = list.iterator(); iter.hasNext(); ) {
       s = (String) iter.next();
       System.out.println(s);
```

```
        }
    }
}
```

Output:

./Demo.java uses unchecked or unsafe operations.

Recompile with -Xlint: unchecked for details.

**Explanation:** In the above example, we are using raw types, it simply means we are not specifying the type of data the list will hold. In this case, the compiler will throw a warning: GenericsErasure.java uses unchecked or unsafe operations. The reason for the warning is that raw types do not carry any type information.

### Issues Caused by Type Erasure

**The issues caused by Type Erasure are listed below:**

The JVM loses information about the generic type at runtime, which is why we can not use instanceof or create new instances of generic types.

- When we work with raw types, the compiler will give a warning.
- We can not create an array of generic types directly. T[] arr = new T[10];
- Method Overloading Issues -
  void print(List<String> list) { }

void print(List<Integer> list) { }
***After erasure, both become:***
**void print(List list)**

**Think of generics like labels on boxes:**

- At compile time → labeled: Box<String>
- At runtime → label removed → just Box

Java ensures safety **before execution**, then removes labels.

## Limitations of Generics

1. **Cannot Create Generic Array**

Invalid:

```
T[] arr = new T[10];
```

2. **Cannot Use instanceof**

   Invalid:

```
if(obj instanceof List<String>)
```

3. **Cannot Use Primitive Types**

   Invalid:

```
List<int>
```

# Tasks to practice on Generics

**1. Generic Class Implementation**

Create a generic class Box<T> that can store and return a value of any type.

**Task:**

- Add methods set(T value) and get()
- Test with Integer, String, and Double

**2. Generic Method**

Write a generic method that prints elements of any array.

**Example:**

printArray(new Integer[]{1,2,3});

printArray(new String[]{"A","B","C"});

**3. Generic Swap Function**

Create a generic method to swap two elements in an array.

**4. Count Occurrences**

Write a generic method that counts how many times a given element appears in a list.

**5. Bounded Type Parameter**

Create a generic class Calculator<T extends Number> with a method:

double sum(List<T> list)

that returns the sum of elements.

**6. Wildcard Usage (? extends)**

Write a method:

void printList(List<? extends Number> list)

that prints elements.

**7. Wildcard Usage (? super)**

Write a method that adds integers to a list:

void addNumbers(List<? super Integer> list)